PROJECT REPORT & PROPOSAL SPECIFICATION

TORNEO: Algorithmic Tournament Management System

Bridging Full-Stack Sports Engineering with Core Data Structures & Algorithms (DSA)

Project Domain: Web Engineering & Applied DSA	**Target Roles:** Admin, Organizer, Viewer
Frontend: React 19, TypeScript, Vite, Tailwind/CSS	**Backend:** FastAPI (Python 3.13), SQLite, SQLAlchemy
Core DSA: Binary Trees, Graphs (DAG), Sorting, Binary Search	**Curriculum Outcomes:** CO3, CO4, CO5 (Computer Science)

1. Executive Summary & Project Novelty

Modern sports tournament websites typically function as thin wrappers around database tables, performing standard SELECT and ORDER BY queries to display brackets and standings. Conversely, academic Data Structures and Algorithms (DSA) courses often teach trees, graphs, and sorting through isolated, terminal-based toy scripts with static dummy arrays.

Torneo solves this duality through a unified architecture: It is a production-ready, full-stack tournament management platform where **classical data structures and algorithmic models serve as the foundational execution engine**. The system manages single-elimination knockouts, round-robin leagues, dynamic bracket propagation, automated multi-variable standings, and features an interactive **DSA Visualization Lab** for real-time educational inspection.

- **1. Explicit Binary Tournament Tree:** Implements a genuine binary tree rooted at the Grand Final. Knockout advancement is an *O(log N)* upward tree traversal with dynamic bye assignment for non-power-of-two team counts.
- **2. Directed Acyclic Graph (DAG) for Match Dependencies:** Models match prerequisites using Adjacency Lists and Matrices. Traverses rounds with BFS and traces championship paths with DFS.
- **3. Multi-Criteria Instrumented Sorting:** Standings are generated via an instrumented Bubble Sort implementing multi-criteria tie-breaking (Points → Goal Diff → Goals Scored → Team Name) with complete execution tracing.
- **4. Interactive DSA Laboratory:** Built-in 'glass-box' visualization engine where students and teachers can step through algorithm passes, inspect Big-O complexities, and observe real-time comparisons and swaps.
- **5. Multi-Format Scheduling:** Full support for both Single Elimination and Round Robin with authentic sporting rules (draws award 1 point without penalty shootouts; knockouts enforce penalty resolution).

2. System Architecture & Technical Stack

Torneo is engineered using a decoupled, client-server REST architecture providing high throughput, type safety, and real-time state responsiveness.

Layer	Technologies Used	Role & Key Responsibilities

Frontend UI	React 19, TypeScript, Vite, TailwindCSS, Lucide Icons, React Flow	Single Page Application (SPA), state management, interactive bracket visualization, responsive dashboards, DSA Lab step visualizer.
Backend API	FastAPI (Python 3.13), Uvicorn, Pydantic v2	High-performance asynchronous REST endpoints, request validation, serialization, business logic routing, audit logging.
DSA Engine	Pure Python (Zero 3rd-party library shortcuts)	Custom implementations of Binary Tournament Tree, Graph Traversal (BFS/DFS), Multi-Criteria Bubble Sort, Binary Search.
Database	SQLite / PostgreSQL, SQLAlchemy 2.0 ORM	Relational data modeling, foreign key constraints, cascading deletions, ACID transaction guarantees for match state updates.
Security	OAuth2, JWT (JSON Web Tokens), bcrypt	Role-Based Access Control (Admin, Organizer, Viewer), password hashing, stateless token authentication.

3. Core Data Structures & Algorithms (DSA) Implementation

A central requirement of the academic curriculum is understanding and applying core data structures (Trees, Graphs, Sorting, Searching). Below is the comprehensive technical breakdown of how each algorithm is mapped and coded in Torneo.

3.1 Binary Tournament Tree (Knockout Elimination)

Source File: backend/app/algorithms/tournament_tree.py | Curriculum Outcome: CO4 (Tree Structures)

Knockout tournaments naturally follow a hierarchical binary tree structure rooted at the championship match (Round *K*). Each match in round *r* is fed by exactly two matches from round *r-1*.

- **Node Definition (MatchNode):** Contains match_id, round_number, scores, winner_name, pointers left (home feeder), right (away feeder), and parent (next round match).
- **Winner Advancement Algorithm:** When a match score is submitted, the tree updates the winner and traverses up the parent pointer to insert the winner into the home or away slot of the subsequent match. This takes **O(log N)** time.
- **Bye Assignment Logic:** For *N* teams where *N* is not a power of 2, the algorithm determines the nearest power of 2 (*2^k ≥ N*) and awards *2^k - N* byes to top seeds, ensuring balanced subtrees.

3.2 Tournament Graph & Dependency Traversal

Source File: backend/app/algorithms/graph_algorithms.py | Curriculum Outcome: CO5 (Graph Structures)

A multi-stage tournament represents a **Directed Acyclic Graph (DAG)**. Vertices *V* represent individual match fixtures, and directed edges *E = (u, v)* indicate that the outcome of match *u* is required before match *v* can commence.

- **Dual Representation:** The graph maintains both an **Adjacency List** (memory-efficient *O(V + E)* representation for sparse brackets) and an **Adjacency Matrix** (constant-time *O(1)* feeder relationship lookup).
- **Breadth-First Search (BFS):** Traverses the tournament round-by-round using a FIFO queue. Used to evaluate match readiness, determine which matches can be scheduled simultaneously across venues, and calculate topological stages.
- **Depth-First Search (DFS):** Traces the lineage of a team backward from the Final node to their qualifying preliminary match, reconstructing their 'path to glory' with *O(V + E)* time complexity.

3.3 Multi-Criteria Bubble Sort with Trace Instrumentor

Source File: backend/app/algorithms/bubble_sort.py | Curriculum Outcome: CO3 (Sorting & Order)

To satisfy the academic requirement of manual sorting without reliance on Python's native sort(), Torneo features a custom multi-pass Bubble Sort equipped with sports tie-breaking logic and an execution tracer.

- **Comparison Hierarchy:** 1) Points (descending) → 2) Goal Difference (descending) → 3) Goals Scored (descending) → 4) Lexicographical Team Name (tie-breaker).
- **State Machine Tracing:** The algorithm captures an array of snapshot frames: {pass, step, compared_indices, swapped, snapshot}. This allows the frontend DSA Lab to animate every swap in real time.
- **Complexity Profile:** Worst-case comparisons: *N(N-1)/2 ∈ O(N^2)*. Optimized with an early-exit flag when zero swaps occur in a pass (best-case *O(N)*).

3.4 Binary Search for Fast Record Lookup

Source File: backend/app/algorithms/binary_search.py | Curriculum Outcome: CO3 (Search Algorithms)

Performs logarithmic *O(log N)* retrieval on pre-sorted team tables and schedules. Records low, mid, and high pointers at every iteration for educational step-through in the DSA visualizer.

3.5 Berger Cyclic Round-Robin Scheduling Algorithm

Source File: backend/app/services/bracket_service.py

Implements the polygon/circle method for league tournament generation: for *N* teams, *N-1* rounds are generated (where *N* is even), rotating *N-1* elements while keeping one fixed. Guarantees that every team plays every other team once with optimal home/away equity.

4. Backend Workflow & Database Architecture

The backend is structured into modular layers adhering to clean architecture principles: **API Endpoints**, **Domain Models**, **Pydantic Schemas**, and **Algorithmic Services**.

4.1 Relational Data Models

Model Name	Key Attributes	Relationships & Constraints
User	id, email, password_hash, role, full_name	Roles: admin, organizer, viewer. Has many tournaments.
Tournament	id, name, sport_type, format, status, max_teams	Format: single_elimination, round_robin. Status: registration_open, in_progress, completed.
Team	id, name, short_name, logo_url, captain_name	Associated via TournamentTeam join-table with tournament-specific stats.
Match	id, tournament_id, round_number, home_score, away_score, winner_id, parent_match_id	Self-referencing tree pointers (parent_match_id, feeder_matches) and venue assignment.
Venue	id, name, address, capacity, surface_type	Hosts match fixtures; prevents double-booking conflicts.
AuditLog	id, user_id, action, entity_type, timestamp	Tracks all administrative actions and score modifications for regulatory auditing.

4.2 Tournament Execution State Machine

1. Setup & Registration: Tournament created with sport rules and venue availability. Teams register until max_teams reached.
2. Schedule Generation: Organizer triggers schedule creation. For Single Elimination, the Binary Tournament Tree generates bracket fixtures with byes. For Round Robin, the Berger cyclic algorithm generates matchday schedules.
3. Match Resolution: When scores are entered, the system validates rules (in knockouts, draws require penalties; in round-robin, draws are preserved awarding 1 point to each team). Winners propagate automatically.
4. Standings Recalculation: Automated service recalculates points, goal differences, and form tables dynamically after each confirmed score.

5. User Interface Showcase & Screen Descriptions

The following screenshots illustrate the actual running Torneo web application, demonstrating both the core tournament operations and the specialized DSA visualization suite.

5.1 Analytics & Tournament Operations Dashboard

Displays real-time tournament metrics, active matches, quick actions, team distribution, and recent score updates.

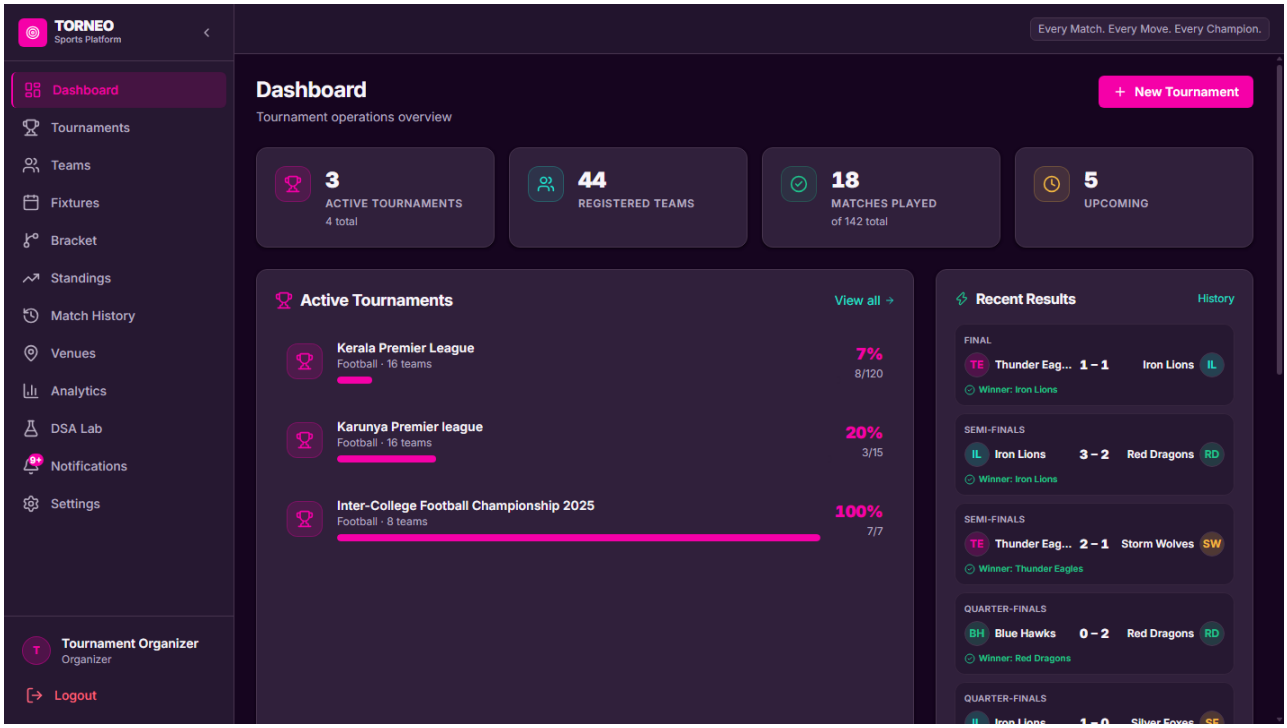


Figure: Actual screenshot of 5.1 Analytics & Tournament Operations Dashboard in Torneo

5.2 Tournament Control Center (Kerala Premier League)

Shows tournament status, format specification, matchday fixture generation, and tab navigation (Fixtures, Bracket/Schedule, Standings, Teams, Venues).

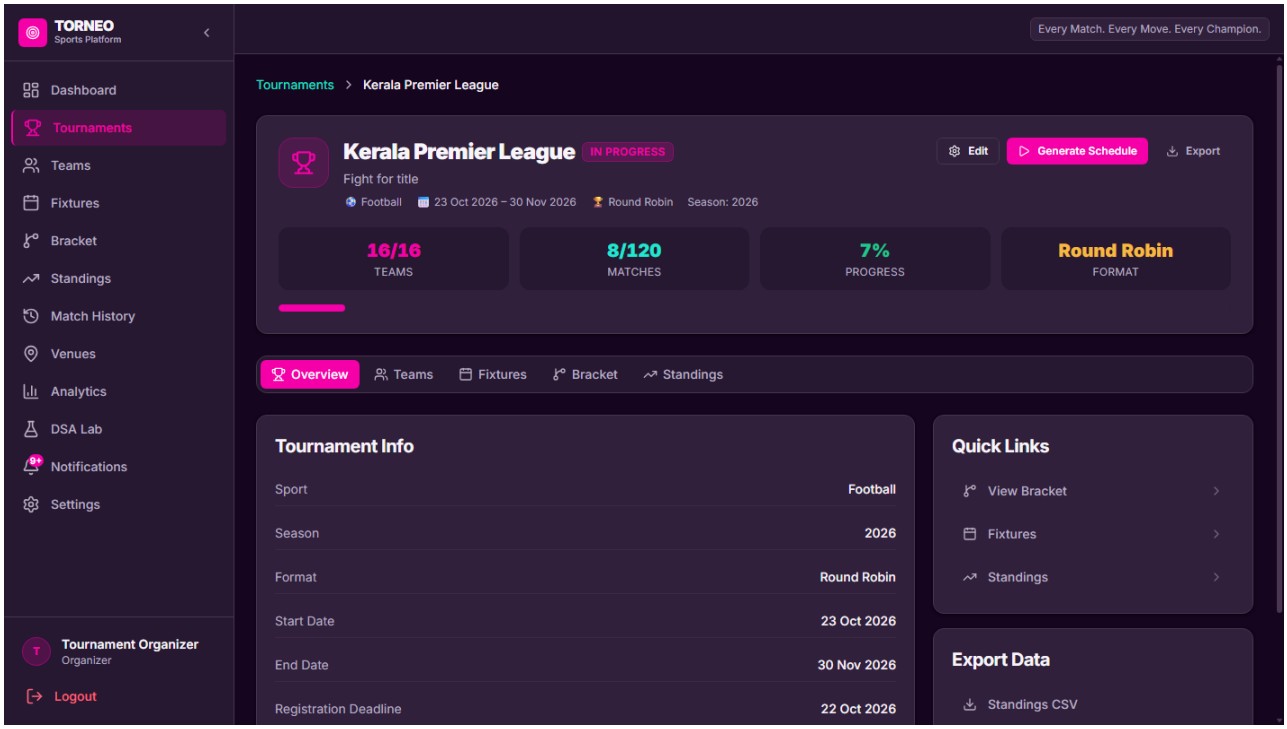


Figure: Actual screenshot of 5.2 Tournament Control Center (Kerala Premier League) in Torneo

5.3 Real-Time Standings & League Table

Live standings calculated dynamically based on match results, displaying Played (P), Won (W), Drawn (D), Lost (L), Goals For (GF), Goals Against (GA), Goal Difference (GD), and Points (PTS).

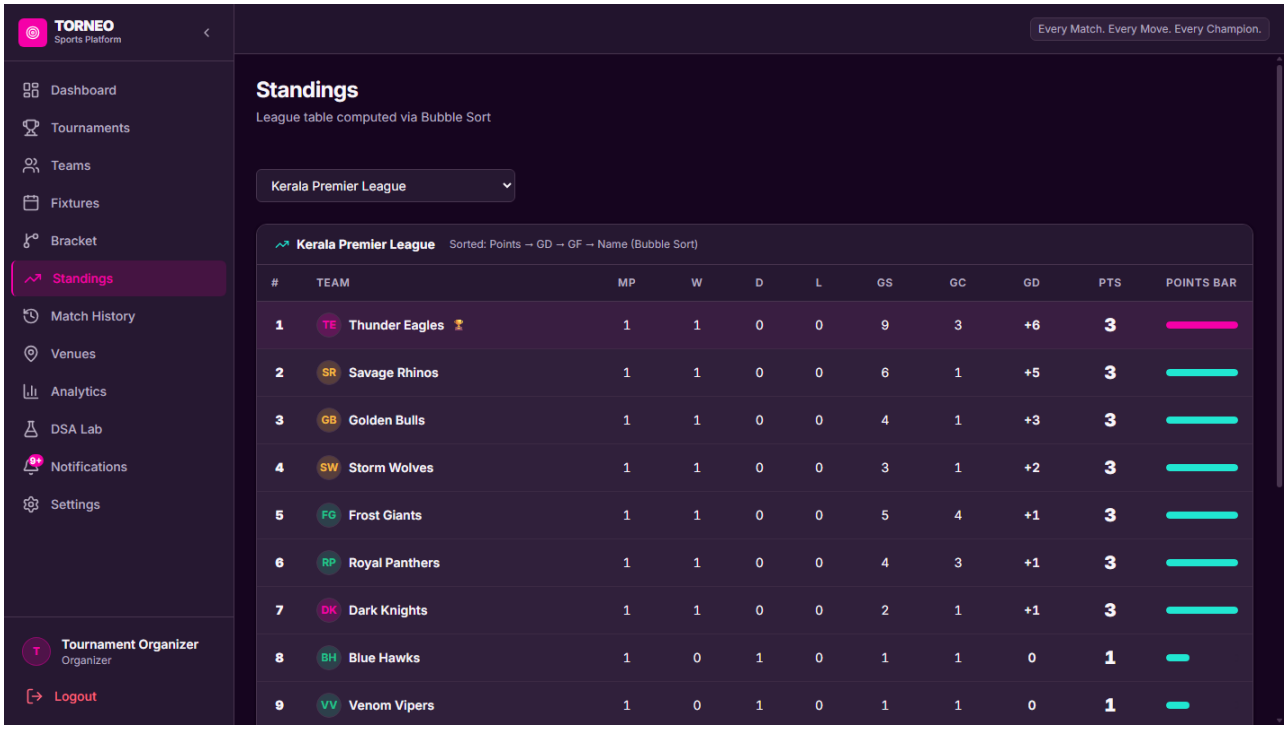


Figure: Actual screenshot of 5.3 Real-Time Standings & League Table in Torneo

5.4 Interactive DSA Lab: Multi-Criteria Bubble Sort Visualizer

Step-by-step visual execution of Bubble Sort on real team standings. Shows comparison counters, swap metrics, O(N^2) complexity indicator, and prev/next playback controls.

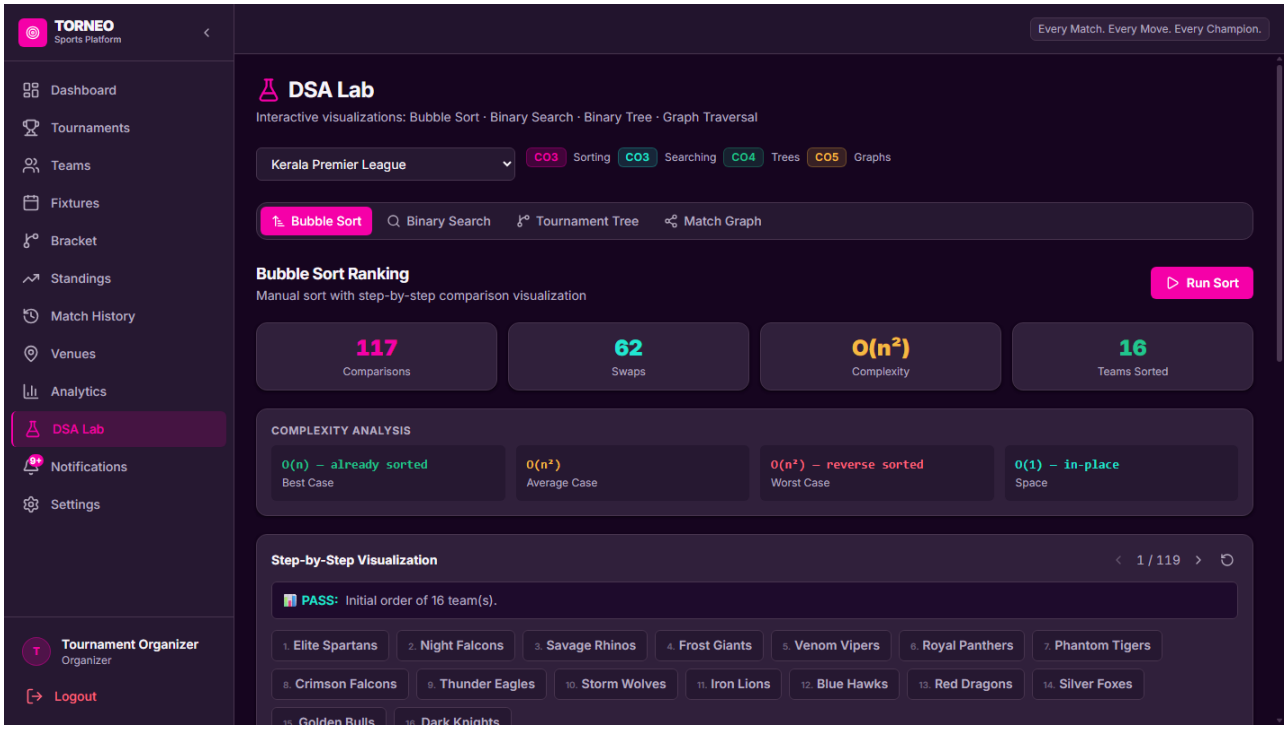


Figure: Actual screenshot of 5.4 Interactive DSA Lab: Multi-Criteria Bubble Sort Visualizer in Torneo

5.5 Interactive DSA Lab: Binary Tournament Tree

Visualizes the explicit binary tree structure of knockout tournaments, showing root (Final), feeder leaf nodes, match progression, and tree depth.

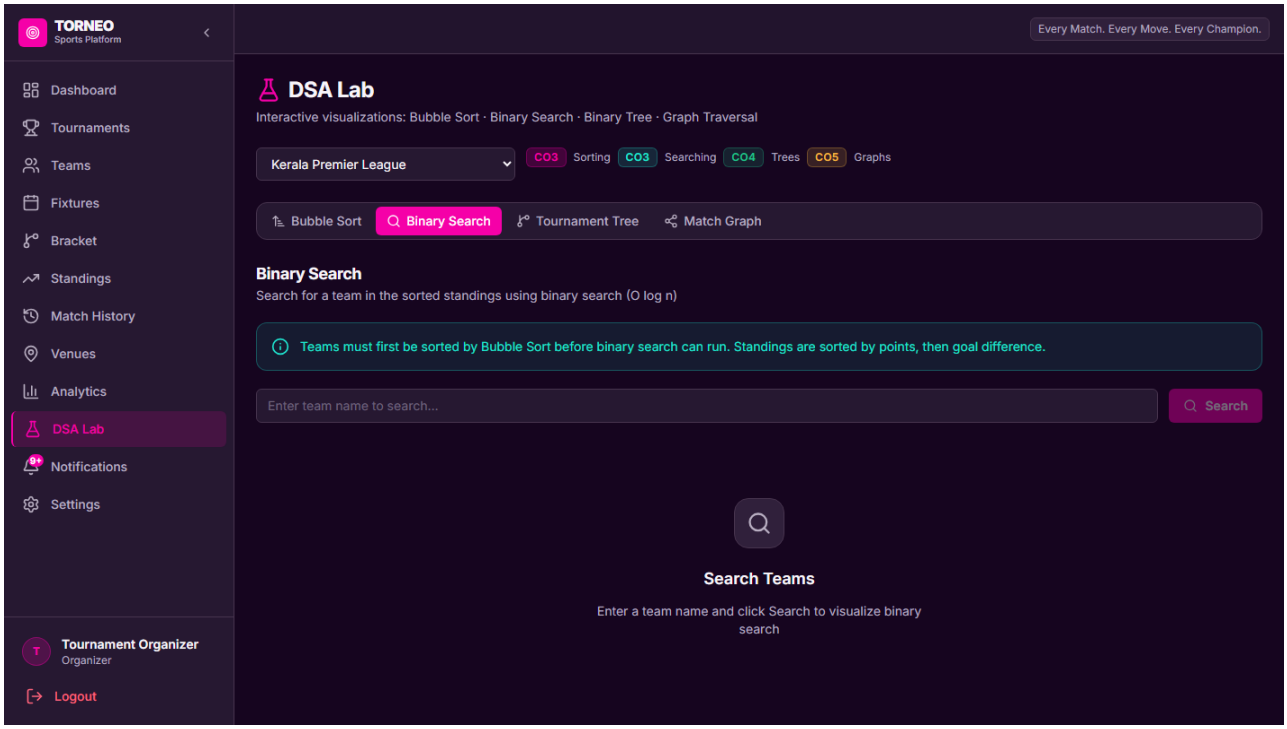


Figure: Actual screenshot of 5.5 Interactive DSA Lab: Binary Tournament Tree in Torneo

5.6 Interactive DSA Lab: Directed Match Progression Graph (DAG)

Visualizes match fixtures as graph vertices and feeder progressions as directed edges. Demonstrates BFS level-order traversal and DFS championship path tracing.

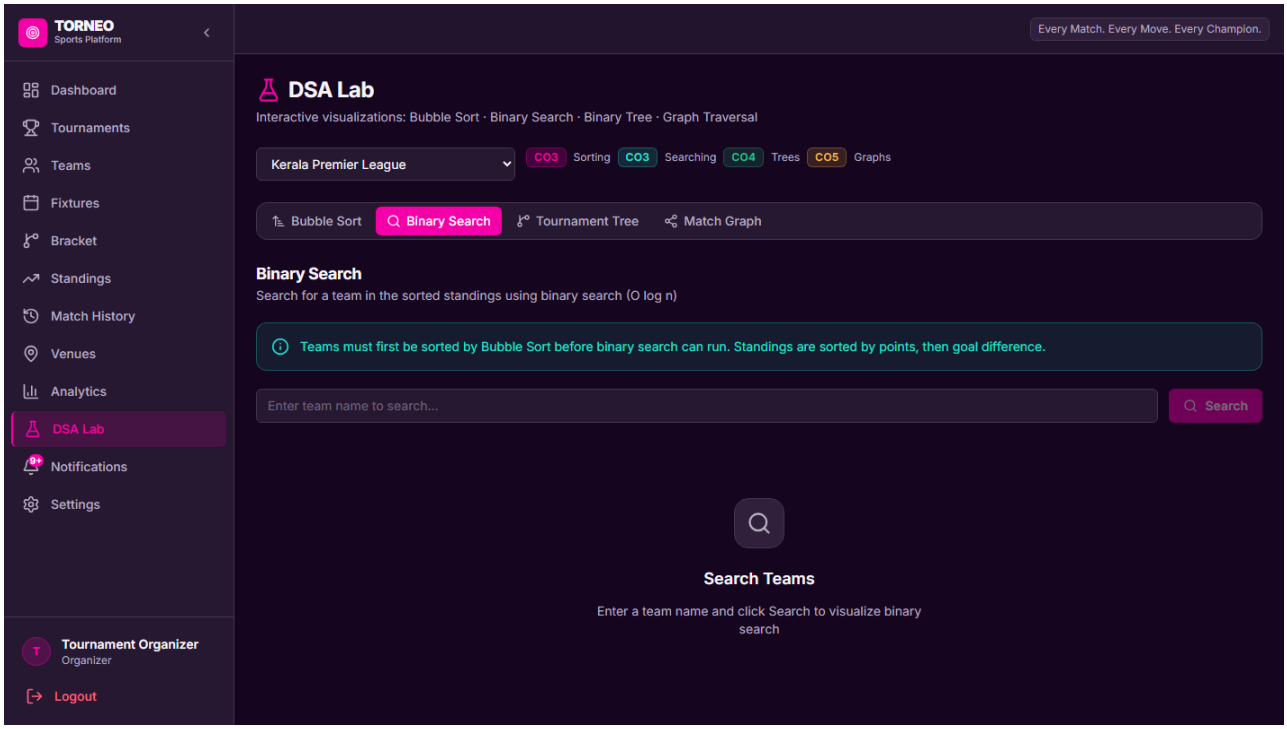


Figure: Actual screenshot of 5.6 Interactive DSA Lab: Directed Match Progression Graph (DAG) in Torneo

6. Ready-to-Use PPT Slide Deck Structure

Use this exact slide-by-slide outline for your project presentation to your teacher/evaluator:

Slide 1: Title & Introduction

Title: 'Torneo - Algorithmic Tournament Management Platform'
Subtitle: Applying Core Data Structures & Algorithms to Modern Web Engineering
Presenter Name, Department of Computer Science & Engineering.

Slide 2: Problem Statement & Motivation

- Traditional tournament platforms lack algorithmic transparency.
- Academic DSA courses teach algorithms in abstract consoles with dummy numbers.
- Need for a production-grade system where real algorithms drive business logic.

Slide 3: Proposed Solution & Key Objectives

- Full-stack management for Knockouts (Single Elimination) & Leagues (Round Robin).
- Implementation of foundational DSA from scratch without built-in library cheats.
- Interactive DSA Laboratory for pedagogical inspection and algorithmic verification.

Slide 4: Technical Architecture & Stack

- Client: React 19, TypeScript, Vite, TailwindCSS (SPA architecture).
- Server: FastAPI, Python 3.13, Uvicorn, SQLAlchemy ORM.
- Database: SQLite / PostgreSQL with foreign key integrity and audit logs.

Slide 5: Algorithm 1 - Binary Tournament Tree (CO4)

- Single elimination modeled as a binary tree rooted at the Final.
- Node structure: MatchNode with left/right feeders and parent pointer.
- O(log N) winner propagation and dynamic bye calculation for non-power-of-2 teams.

Slide 6: Algorithm 2 - Tournament DAG & Graph Traversal (CO5)

- Tournament match dependencies modeled as Directed Acyclic Graph.
- Adjacency List & Matrix representations.
- BFS for concurrent round scheduling; DFS for championship path lineage.

Slide 7: Algorithm 3 - Multi-Criteria Bubble Sort (CO3)

- Standings sorting using Points → Goal Diff → Goals Scored → Team Name.
- Instrumented execution tracing for step-by-step playback in the UI.
- Worst-case O(N^2) vs Best-case O(N) early termination.

Slide 8: Algorithm 4 - Round Robin Scheduling & Draw Handling

- Berger cyclic polygon algorithm for fair all-play-all schedule generation.
- Clear distinction between knockout tie-breakers (penalties) vs league ties (1 pt each).

Slide 9: Live Demo & DSA Lab Showcase

- Walkthrough of Tournament Creation & Schedule Generation.
- Live demonstration of the DSA Lab: stepping through Bubble Sort, Tree, and Graph tabs.

Slide 10: Conclusion, Outcomes & Future Enhancements

- Achieved: Unified production app with academic DSA rigor.
- Course Outcomes validated: CO3 (Sorting/Searching), CO4 (Trees), CO5 (Graphs).
- Future work: Double elimination brackets, Swiss-system format, live push notifications.

7. Examiner / Viva Defense Q&A; Guide

Prepare for your presentation and viva evaluation with these answers to expected faculty questions:

Q1: Why did you implement Bubble Sort instead of using Python's built-in sorted() or QuickSort?

Answer: While built-in sorted() is faster (Timsort, *O(N log N)*), it acts as a black box that cannot be inspected step-by-step. We implemented our own multi-criteria sorting algorithm with execution instrumentation so that every pass, comparison, and swap is recorded and can be played back visually in the DSA Lab for academic verification.

Q2: How does the Binary Tournament Tree handle a tournament with 5 or 6 teams?

Answer: A balanced binary tree requires a power of 2 number of leaf slots (e.g., 4, 8, 16). When *N=6* teams participate, the algorithm calculates the next power of 2 (*8*), awards *8 - 6 = 2* byes to the highest seeds, and places the remaining 4 teams in preliminary matches, seamlessly preserving the binary tree invariant.

Q3: What is the time complexity of propagating a winner through the tournament bracket?

Answer: Because the tournament bracket is structured as a balanced binary tree of depth *D = log_2(N)*, winner advancement simply follows the parent pointer from the completed match to the root. Thus, updating the bracket takes exactly **O(log N)** operations.

Q4: How are draw matches handled differently between Single Elimination and Round Robin?

Answer: In single elimination knockouts, a match cannot end in a draw because a winner is required to advance up the binary tree; therefore, the system mandates a penalty shootout. In round robin league format, draws are a valid terminal state; both teams are awarded 1 point and winner_id is set to None.

Q5: How is graph traversal applied in this project?

Answer: Matches are modeled as vertices in a Directed Acyclic Graph (DAG) with edges representing feeder progression. Breadth-First Search (BFS) is used to traverse matches level-by-level to ensure preceding matches are complete before scheduling dependent matches. Depth-First Search (DFS) backtracks from the final to trace any team's route through the tournament.